

Practical: 6	Implementation of chain matrix multiplication using dynamic programming
01/09/2026	

Aim: To Implement the chain matrix multiplication using dynamic programming

Algorithm:

Algorithm:-

Input : n matrices and dimension array
 $P[0, \dots, n]$

Output : Minimum no-of scalar multiplications required to multiply all matrices

steps:

- 1, start
- 2, Read the no-of matrices n
- 3, Read the dimension array
 $P[0], P[1], \dots, P[n]$
- 4, create two tables
 $C[n+1][n+1]$
 $K[n+1][n+1]$
- 5, Initialize the cost table for a single matrix
 $C[i][i] = 0$
- 6, $d=1$ to $n-1$ Repeat in for loop
 calculate the ending matrix
 $j = i + d$
- 7, for $x=i$ where $x < j$ do
 $q = C[i][x] + C[x+1][j] + P[i-1] \times P[x] \times P[j]$
- 8, if $(min > q)$ then do
 $min = q$;
 $K[i][j] = x$;
- 9, increment the 'x' values
- 10, Assign $C[i][j] = min$
- 11, Stop

Program:

```
#include <iostream>
using namespace std;

int main(){

    int n = 5;

    int p[] = {5, 10, 3, 12, 5, 50};

    int c[10][10] = {0};

    int k[10][10] = {0};

    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= n; j++){
            intj=i+d;
            int min=99999;

            for(int x = i; x < j; x++){
                intq=c[i][x]+c[x+1][j]
                +p[i-1]*p[x]*p[j];

                if(q < min){
                    min=q;
                    k[i][j] = x;
                }
            }
            c[i][j] = min;
        }
    }

    cout<<"\nMinimumcost="<< c[1][n]<<endl;

    return 0;
}
```

Output:

```
C:\Users\Chandu\OneDrive\D x + v

Minimum cost = 1655

-----
Process exited after 0.01228 seconds with return value 0
Press any key to continue . . . |
```

Time Complexity:

time complexity :-

```

for(d = 1; d < n; d++) {          → n
    for(i = 1; i < n-d; i++) {      → n-d
        j = i+d;
        for(k = i; k <= j-1; k++) { → d
            q = min[i][k] + min[k+1][j] + p[i-1]*A[k]*A[j];
            if (q < min[i][j])
                min[i][j] = q;
        }
    }
}

```

$T(n) = \sum_{d=1}^{n-1} (n-d) \times d$
 $= \sum_{d=1}^{n-1} (nd - d^2)$
 $= n \sum_{d=1}^{n-1} d - \sum_{d=1}^{n-1} d^2$
 $T(n) = n \left[\frac{n(n-1)}{2} \right] - \frac{n(n-1)(2n-1)}{6}$
 $= \frac{3n^2(n-1) - n(n-1)(2n-1)}{6}$
 $= \frac{n(n-1)(n+1)}{6}$
 $= \frac{(n^3 - n)}{6}$
 $= O(n^3)$

Best case = $O(n^3)$
 Average case = $O(n^3)$
 Worst case = $O(n^3)$
 Space complexity = $O(n^2)$

Conclusion:

The Implementation of the of chain matrix multiplication using dynamic programming has successfully executed with the time complexity of $O(n^3)$ in the Three cases of best, average and worst. At the end of this program says that the Minimum number of scalar multiplications required to multiply all matrices.